create-student-account-final.ts

UNIGATES - Secure Student Account Creation Edge Function

Generated: 2026-07-29

```
import { createClient } from "https://esm.sh/@supabase/supabase-js@2.45.0";

const ALLOWED_ORIGINS = [
  "https://unigates.online",
  "https://www.unigates.online",
  "https://preview--unigates-online.lovable.app",
  "http://localhost:5173",
];

function isOriginAllowed(origin: string): boolean {
  return ALLOWED_ORIGINS.some((p) => origin === p);
}

function getCorsHeaders(req: Request): Record<string, string> {
  const origin = req.headers.get("origin") || "";
  const allowed = isOriginAllowed(origin);
  return {
    "Access-Control-Allow-Origin": allowed ? origin : "null",
    "Access-Control-Allow-Headers": "authorization, x-client-info, apikey, content-type",
    "Access-Control-Allow-Methods": "POST, OPTIONS",
  };
}

const corsHeaders = getCorsHeaders(new Request("http://localhost"));

function genPassword() {
  const alpha = "ABCDEFGHJKLMNPQRSTUVWXYZabcdefghijkmnopqrstuvwxyz";
  const digits = "23456789";
  const all = alpha + digits + "!@#$%^&*";
  const buf = new Uint32Array(16);
  crypto.getRandomValues(buf);
  let out = "";
  for (let i = 0; i < 16; i++) out += all[buf[i] % all.length];
  const d = new Uint32Array(2);
  crypto.getRandomValues(d);
  out = alpha[d[0] % alpha.length] + digits[d[1] % digits.length] + out.slice(2);
  return out;
}

async function hashIp(ip: string): Promise<string> {
  const data = new TextEncoder().encode(ip);
  const hash = await crypto.subtle.digest("SHA-256", data);
  return Array.from(new Uint8Array(hash))
    .map((b) => b.toString(16).padStart(2, "0"))
    .join("");
}

const RATE_LIMIT_WINDOW_MS = 60_000;
const RATE_LIMIT_MAX_ATTEMPTS = 5;

async function checkRateLimit(admin: ReturnType<typeof createClient>, ipHash: string, userId?: string): Promise<void> {
  const cutoff = new Date(Date.now() - RATE_LIMIT_WINDOW_MS).toISOString();

  let total = 0;

  const { count: ipCount, error: ipErr } = await admin
    .from("invitation_rate_limits")
    .select("*", { count: "exact", head: true })
    .eq("ip_hash", ipHash)
    .gte("attempted_at", cutoff);

  if (ipErr) {
    console.error("[create-student-account] checkRateLimit ip query error:", ipErr.message);
    throw new ServerError("Rate limit check failed");
  }

  total += ipCount || 0;

  if (userId) {
    const { count: userCount, error: userErr } = await admin
      .from("invitation_rate_limits")
      .select("*", { count: "exact", head: true })
      .eq("user_id", userId)
      .gte("attempted_at", cutoff);

    if (userErr) {
      console.error("[create-student-account] checkRateLimit user query error:", userErr.message);
      throw new ServerError("Rate limit check failed");
    }

    total += userCount || 0;
  }

  if (total >= RATE_LIMIT_MAX_ATTEMPTS) {
    throw new RateLimitError("Too many attempts. Please try again later.");
  }
}

class RateLimitError extends Error {
  status = 429;
}

class ServerError extends Error {
  status = 503;
}

async function recordAttempt(admin: ReturnType<typeof createClient>, ipHash: string, userId?: string): Promise<void> {
  const { error } = await admin.from("invitation_rate_limits").insert({
    ip_hash: ipHash,
    user_id: userId || null,
    action_type: "create_student_account",
  });

  if (error) {
    console.error("[create-student-account] recordAttempt insert error:", error.message);
    throw new ServerError("Rate limit recording failed");
  }
}

const EMAIL_RE = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

function validateEmail(v: string): boolean {
  return EMAIL_RE.test(v);
}

function json(b: unknown, s = 200, extraHeaders: Record<string, string> = {}) {
  return new Response(JSON.stringify(b), {
    status: s,
    headers: { ...corsHeaders, "Content-Type": "application/json", ...extraHeaders },
  });
}

function fail(step: string, message: string, status = 500) {
  console.error(`[create-student-account] step "${step}" failed: ${message}`);
  return json({ error: "Account creation failed" }, status);
}

function isEmailAlreadyExistsError(error: any) {
  const code = String(error?.code || "").toLowerCase();
  const message = String(error?.message || "").toLowerCase();
  return (
    code === "email_exists" ||
    message.includes("already registered") ||
    message.includes("already exists") ||
    message.includes("user already") ||
    message.includes("email is already") ||
    message.includes("email already")
  );
}

function validatePassword(pw: string): { ok: boolean; reason?: string } {
  if (!pw || typeof pw !== "string") return { ok: false, reason: "Password is required" };
  if (pw !== pw.trim()) return { ok: false, reason: "Password must not have leading or trailing spaces" };
  if (pw.length < 8) return { ok: false, reason: "Password must be at least 8 characters" };
  if (!/[A-Z]/.test(pw)) return { ok: false, reason: "Password must contain an uppercase letter" };
  if (!/[a-z]/.test(pw)) return { ok: false, reason: "Password must contain a lowercase letter" };
  if (!/\d/.test(pw)) return { ok: false, reason: "Password must contain a digit" };
  if (/\s/.test(pw)) return { ok: false, reason: "Password must not contain spaces" };
  return { ok: true };
}

async function rollbackNewUser(admin: ReturnType<typeof createClient>, uid: string) {
  const tables = ["department_members", "college_members", "user_roles", "profiles"] as const;
  for (const table of tables) {
    const { error } = await admin.from(table).delete().eq("user_id", uid);
    if (error) {
      console.error(`[create-student-account] rollback failed for ${table}: ${error.message}`);
    }
  }
  const { error } = await admin.auth.admin.deleteUser(uid);
  if (error) {
    console.error(`[create-student-account] rollback failed to delete auth user ${uid}: ${error.message}`);
  }
}

const MAX_BODY_BYTES = 65536;

Deno.serve(async (req) => {
  const headers = getCorsHeaders(req);

  if (req.method === "OPTIONS") return new Response(null, { headers });

  if (req.method !== "POST") {
    return json({ error: "Method not allowed" }, 405, headers);
  }

  try {
    let parsed: unknown;
    try {
      const text = await req.text();
      if (new TextEncoder().encode(text).length > MAX_BODY_BYTES) {
        return json({ error: "Request body too large" }, 413, headers);
      }
      parsed = JSON.parse(text);
    } catch {
      return json({ error: "Invalid JSON" }, 400, headers);
    }

    if (!parsed || typeof parsed !== "object" || Array.isArray(parsed)) {
      return json({ error: "Request body must be a JSON object" }, 400, headers);
    }
    const body = parsed as Record<string, unknown>;

    const fullName = String(body.full_name || "").trim();
    if (!fullName) return json({ error: "Missing full_name" }, 400, headers);
    if (fullName.length > 200) return json({ error: "full_name too long" }, 400, headers);

    let email = String(body.email || "")
      .trim()
      .toLowerCase();
    if (email && email.length > 254) return json({ error: "email too long" }, 400, headers);
    if (email && !validateEmail(email)) {
      return json({ error: "Invalid email format" }, 400, headers);
    }

    const phone = String(body.phone || "").trim();
    if (phone.length > 30) return json({ error: "phone too long" }, 400, headers);

    const rawCodes = body.department_codes;
    if (!Array.isArray(rawCodes)) {
      return json({ error: "department_codes must be an array" }, 400, headers);
    }
    if (rawCodes.length > 50) {
      return json({ error: "Too many department codes" }, 400, headers);
    }
    const codes: string[] = Array.from(
      new Set(rawCodes.map((c: string) => String(c).trim().toUpperCase()).filter(Boolean)),
    );
    if (codes.length === 0) return json({ error: "Missing department codes" }, 400, headers);

    const invitationCode = String(body.invitation_code || "").trim();
    if (invitationCode.length > 100) return json({ error: "invitation_code too long" }, 400, headers);

    const temporaryPassword = String(body.temporary_password || "");
    if (temporaryPassword.length > 128) return json({ error: "temporary_password too long" }, 400, headers);

    const admin = createClient(Deno.env.get("SUPABASE_URL")!, Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!, {
      auth: { autoRefreshToken: false, persistSession: false },
    });

    const rawIp = (req.headers.get("x-forwarded-for") || req.headers.get("cf-connecting-ip") || "unknown")
      .split(",")[0]
      .trim();
    const ipHash = await hashIp(rawIp);

    let institutionId: string;
    let institutionType: "college" | "institute";
    let callerUserId: string | undefined;

    // %% Invitation flow %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
    if (invitationCode) {
      try {
        await checkRateLimit(admin, ipHash);
      } catch (e: any) {
        return json({ error: e.message }, e.status || 429, headers);
  }

  const pwCheck = validatePassword(temporaryPassword);
  if (!pwCheck.ok) {
    return json({ error: pwCheck.reason }, 400, headers);
  }

  try {
    await recordAttempt(admin, ipHash);
  } catch {
    return json({ error: "Too many attempts. Please try again later." }, 503, headers);
  }

  const { data: collegeInv, error: collegeErr } = await admin
    .from("college_invitations")
    .select("role, college_id")
    .eq("invitation_code", invitationCode)
    .eq("is_active", true)
    .or("expires_at.is.null,expires_at.gt.now()")
    .maybeSingle();

  if (collegeErr) return fail("invitation_lookup", "Failed to validate invitation", 500);

  if (collegeInv) {
    if ((collegeInv.role || "").toLowerCase() !== "student") {
      return json({ error: "This invitation is not for students" }, 400, headers);
    }
    institutionId = collegeInv.college_id;
    institutionType = "college";
  } else {
    const { data: instituteInv, error: instituteErr } = await admin
      .from("institute_invitations")
      .select("role, institute_id")
      .eq("invitation_code", invitationCode)
      .eq("is_active", true)
      .or("expires_at.is.null,expires_at.gt.now()")
      .maybeSingle();

    if (instituteErr) return fail("invitation_lookup", "Failed to validate invitation", 500);

    if (!instituteInv) {
      return json({ error: "Invalid, expired, or already used invitation" }, 400, headers);
    }

    if ((instituteInv.role || "").toLowerCase() !== "student") {
      return json({ error: "This invitation is not for students" }, 400, headers);
    }
    institutionId = instituteInv.institute_id;
    institutionType = "institute";
  }
}
    // %% Admin flow (authenticated institution admin) %%%%%%%%%%%
else {
  const authHeader = req.headers.get("Authorization") || "";
  const userClient = createClient(Deno.env.get("SUPABASE_URL")!, Deno.env.get("SUPABASE_ANON_KEY")!, {
    global: { headers: { Authorization: authHeader } },
  });
  const {
    data: { user },
  } = await userClient.auth.getUser();
  if (!user?.email) return json({ error: "Unauthorized" }, 401, headers);

  callerUserId = user.id;

  try {
    await checkRateLimit(admin, ipHash, callerUserId);
  } catch (e: any) {
    return json({ error: e.message }, e.status || 429, headers);
  }

  try {
    await recordAttempt(admin, ipHash, callerUserId);
  } catch {
    return json({ error: "Too many attempts. Please try again later." }, 503, headers);
  }

  const callerEmail = user.email.toLowerCase();
  const { data: colleges, error: collegeRegErr } = await admin
    .from("college_registrations")
    .select("id, email, status")
    .eq("email", callerEmail)
    .eq("status", "approved");
  if (collegeRegErr) {
    console.error("[create-student-account] college_registrations query error:", collegeRegErr.message);
    return json({ error: "Account creation failed" }, 503, headers);
  }
  const { data: institutes, error: instituteRegErr } = await admin
    .from("institute_registrations")
    .select("id, email, status")
    .eq("email", callerEmail)
    .eq("status", "approved");
  if (instituteRegErr) {
    console.error("[create-student-account] institute_registrations query error:", instituteRegErr.message);
    return json({ error: "Account creation failed" }, 503, headers);
  }

  const ownedColleges = (colleges || []).map((r: any) => ({ id: r.id, type: "college" as const }));
  const ownedInstitutes = (institutes || []).map((r: any) => ({ id: r.id, type: "institute" as const }));
  const owned = [...ownedColleges, ...ownedInstitutes];

  if (owned.length === 0) {
    return json({ error: "No approved institution found for this admin" }, 403, headers);
  }

  if (owned.length === 1) {
    institutionId = owned[0].id;
    institutionType = owned[0].type;
  } else {
    const explicitId = String(body.institution_id || "").trim();
    const match = owned.find((o) => o.id === explicitId);
    if (!match) {
      return json({ error: "Multiple institutions found; specify institution_id" }, 400, headers);
    }
    institutionId = match.id;
    institutionType = match.type;
  }
}

    // %% Resolve departments (exact match required before Auth us
const { data: depts, error: deptErr } = await admin
  .from("departments")
  .select("id, department_code, institution_id, institution_type")
  .in("department_code", codes);

if (deptErr) return fail("resolve_depts", "Failed to resolve departments", 500);

const seenCodes = new Map<string, number>();
for (const d of depts || []) {
  const code = (d.department_code || "").toUpperCase();
  seenCodes.set(code, (seenCodes.get(code) || 0) + 1);
}
const ambiguous = codes.filter((c) => (seenCodes.get(c) || 0) > 1);
if (ambiguous.length > 0) {
  return json({ error: `Ambiguous department codes: ${ambiguous.join(", ")}` }, 400, headers);
}

const matchedCodes = new Set((depts || []).map((d: any) => (d.department_code || "").toUpperCase()));
const missing = codes.filter((c) => !matchedCodes.has(c));
if (missing.length > 0) {
  return json({ error: `Department code(s) not found: ${missing.join(", ")}` }, 404, headers);
}

const allowedDepts = (depts || []).filter(
  (d: any) => d.institution_id === institutionId && d.institution_type === institutionType,
);
if (allowedDepts.length !== codes.length) {
  return json({ error: "Some departments do not belong to the specified institution" }, 403, headers);
}

    // %% Build email if not provided %%%%%%%%%%%%%%%%%%%%%%%%%%%%
if (!email) {
  const slug =
    fullName
      .toLowerCase()
      .replace(/ /g, ".")
      .replace(/[^a-z0-9.]/g, "")
      .slice(0, 20) || "student";
  email = `${slug}.${Date.now().toString(36)}@unigates.online`;
}

const password = invitationCode ? temporaryPassword : genPassword();

    // %% Create user account and all related records %%%%%%%%%%%%
let uid: string | null = null;

try {
  const { data: users } = await admin.auth.admin.listUsers({ page: 1, perPage: 1000 } as any);
  const existingUser = (users?.users || []).find((u: any) => (u.email || "").trim().toLowerCase() === email);
```

```
if (existingUser) {
  return json({ error: "An account with this email already exists" }, 409, headers);
}

const { data: created, error: ce } = await admin.auth.admin.createUser({
  email,
  password,
  email_confirm: true,
  user_metadata: {
    full_name: fullName,
    seeded_via: invitationCode ? "student_invitation" : "institution_admin",
    must_change_password: invitationCode ? false : true,
  },
});

if (ce) {
  if (isEmailAlreadyExistsError(ce)) {
    return json({ error: "An account with this email already exists" }, 409, headers);
  }
  throw ce;
}
if (!created.user) throw new Error("createUser returned no user");

uid = created.user.id;

const { error: pe } = await admin.from("profiles").insert({
  user_id: uid,
  email,
  full_name: fullName,
  phone: phone || null,
  role: "student",
  platform: "courses",
});
if (pe) throw new Error(`profile insert: ${pe.message}`);

const { error: roleErr } = await admin.from("user_roles").insert({
  user_id: uid,
  role: "student" as any,
  approved: true,
});
if (roleErr) throw new Error(`user_roles insert: ${roleErr.message}`);

let joined = 0;
for (const d of allowedDepts) {
  const { error: memberErr } = await admin.from("department_members").insert({
    department_id: d.id,
    full_name: fullName,
    role: "student",
    user_id: uid,
    email,
    phone: phone || null,
  });
  if (memberErr) throw new Error(`department_members insert: ${memberErr.message}`);

  if (d.institution_type === "college") {
    const { error: colMemberErr } = await admin
      .from("college_members")
      .upsert({ college_id: d.institution_id, user_id: uid, role: "student" }, {
        onConflict: "college_id,user_id",
      } as any);
    if (colMemberErr) throw new Error(`college_members upsert: ${colMemberErr.message}`);
  }

  joined++;
}

    // %% Consume invitation AFTER successful account creation %
if (invitationCode) {
  const { data: consumeResult, error: consumeErr } = await admin.rpc("consume_institution_invitation" as any, {
    _code: invitationCode,
  });

  if (consumeErr) {
    throw new Error(`invitation consumption rpc error: ${consumeErr.message}`);
  }

  const invResult = consumeResult?.[0];
  if (!invResult || !invResult.is_valid) {
    throw new Error("invitation already consumed or expired");
  }
}

const response: Record<string, any> = { success: true, email, user_id: uid, joined };
const extraHeaders: Record<string, string> = {};
```

```
      if (!invitationCode) {
        response.password = password;
        extraHeaders["Cache-Control"] = "no-store, no-cache, must-revalidate, private";
      }

      return json(response, 200, { ...headers, ...extraHeaders });
    } catch (e: any) {
      if (uid) {
        await rollbackNewUser(admin, uid);
      }
      return json({ error: "Account creation failed" }, 500, headers);
    }
  } catch (e: unknown) {
    console.error("[create-student-account] unhandled error:", String(e));
    return json({ error: "Account creation failed" }, 500, headers);
  }
});
```